ANSWER KEY · AGENTIC HUNTING WORKSHOP

Module 1 — Hunt the Front Door by Hand (30 min, together)

> We do this together. Type along if you're hands-on; watch the screen if not. **Level coverage: L0 → L1.** You leave with two hunts on disk: **H-0004** (done) and **H-0005** (open).

Setup (5 min)

1. Install ATHF. In a terminal, run this block. It throws away any old venv so everyone is on the same build:

```
deactivate 2>/dev/null
rm -rf .venv
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install 'agentic-threat-hunting-framework[all]'
athf --version
```

You want a version number back from the last line. No version? Flag one of us. No LLM key needed today.

2. Log in to Splunk. Open the credentials sheet, type your name in one free row (one person per row), and open that row's **Splunk URL** in your browser. Log in with that row's username and password, then go to **Search & Reporting**. All the data lives in `index=thrunt`.

3. Set the time picker to Last 30 days. The time picker is the drop-down to the right of the search bar. The attack happened over the past week, so "Last 24 hours" misses most of it — and "Last 7 days" cuts off the very start.

4. Set up your ATHF workspace. In the same terminal:

```
athf init
```

`athf init` asks five quick questions (workspace name, SIEM, EDR, hunt-ID prefix, retention). **Just press Enter through all of them.** The defaults (Splunk, `H-`, and so on) are exactly what this workshop expects. This creates `hunts/`, `AGENTS.md`, and the hunt template. Confirm it worked:

```
athf hunt list
```

Three seeded example hunts (H-0001–H-0003) is expected. Leave them.

If `athf` isn't found, activate the venv from step 1 (`source .venv/bin/activate`), then `athf --version`.

> **Answer key — staff + self-study**
>
> Hint trigger: `athf: command not found` almost always means the venv isn't active in this terminal. `source .venv/bin/activate` fixes it. If Splunk shows "No results" on the first search, check the time picker before anything else.

Part A — Follow the process tree (12 min)

Guiding question (orientation): *The alert says `node.exe` ran right after a package install on `WS-BUILD-01`. Before you search: what would a malicious install do that a normal one wouldn't?*

How lineage works in eCAR. Every process event carries four fields at the top level:

- `pid` is the process itself. `ppid` is the process that launched it (its parent's `pid`).
- `hostname` is where it ran. `principal` is the account it ran as.

So a process's **children** are `ppid=<its pid>`, and its **siblings** share its `ppid`. One catch: Windows reuses PIDs. A PID names one process on one host, and only while that process lives. So every PID search below also says `hostname="WS-BUILD-01"`, and you glance at `_time`.

How you read the results. For A1 and A2, stay in Splunk's default **Events** view and read the event itself: that's what an eCAR record looks like, and it shows you where `pid` and `ppid` sit. From A3 on, the search ends in `| table`, so the rows line up as a timeline with `_time` first.

Moving a PID to the next search. PIDs are short numbers. Write them down and type them over the `<...>` placeholder in the next search.

A1. Start at the alert.

Why this pivot: The alert names node running a post-install script on WS-BUILD-01. Find that process and see who launched it.

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" properties.command_line="*postinstall.js*"
```

Read the event. It's one JSON record: `hostname`, `principal`, `pid` and `ppid` at the top, and the process details under `properties` (`command_line`, `image_path`, `parent_image_path`). You'll also see two long UUID fields; we don't use them. Write down two numbers, labelled: **node's pid** and **node's ppid**.

> **Answer key — staff + self-study**
>
> Expected: one event, `principal` alice.chmod. `node C:\Users\alice.chmod\AppData\npm\packmule-cli\postinstall.js` (`C:\Program Files\nodejs\node.exe`).
>
> - node's `pid` = **5648**
> - node's `ppid` = **5612**. `properties.parent_image_path` is `powershell.exe`: 5612 is alice's PowerShell window (verified: `pid=5612` on WS-BUILD-01 is powershell.exe, created 7s before node). Search A1 lands on the front door (TIER_MANIFEST A1). Model answer (orientation Q): a normal install writes files and exits. A malicious one runs something extra: a post-install script that launches a shell. Name the behavior, not the package.

A2. What did node launch?

Why this pivot: Follow the children: processes on this host whose `ppid` is node's `pid`.

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" ppid=<node's pid>
```

Still the Events view. Check that this event's `ppid` is the number you wrote down.

> **Answer key — staff + self-study**
>
> **Type:** `ppid=5648` Expected: one event, ~2s after node: `cmd.exe /c "packmule --activate-telemetry"`, `pid` 5656. `/c` means "run one command and exit", and nothing else hangs off it (`ppid=5656` returns zero processes). This is where most people stop. Don't.

A3. Every shell on this host, in time order.

Why this pivot: A child that goes nowhere isn't the end. Widen out to every `cmd.exe` on WS-BUILD-01. The last filter keeps the shells that mention `telemetry`, the word in A2's shell. From here on the results are a table.

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" properties.image_path="*\\cmd.exe"
properties.command_line="*telemetry*"
| sort _time
| table _time pid ppid properties.parent_image_path principal
  properties.image_path properties.command_line
```

Compare each row's `ppid` with your A1 numbers; `properties.parent_image_path` names the parent. Which shell did node launch, and which one did node's parent launch?

> **Answer key — staff + self-study**
>
> Expected: two rows, both alice.chmod:
>
> 1. node +2s: `cmd.exe /c "packmule --activate-telemetry"`, pid 5656, **ppid 5648**, parent `C:\Program Files\nodejs\node.exe`. A2's dead end.
> 2. node +134s: `cmd.exe /k "packmule telemetry shell"`, **pid 5668**, **ppid 5612**, parent `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`: the same parent as node. It's node's **sibling**, started by alice's PowerShell, so a search that only follows node's children never finds it. `/k` means "keep the shell open": an interactive shell called "telemetry shell" is the tell. (TIER_MANIFEST A2.) What the `telemetry` filter hides: WS-BUILD-01 has 202 `cmd.exe` launches over the week, mostly alice's dev work (`npm run build/test/dev`, bare `cmd.exe /k`). Two of them are later modules' reveals, so don't show them here: `cmd.exe /k …\packmule\hooks\pre-commit.cmd` ~74 min after node (parent git.exe, M3 persistence) and `cmd.exe /c type … \project\.env` / `…\.aws\credentials` ~2h20m after node (ppid 5612, M4 credential reads). That's why the filter is `*telemetry*`, not `*packmule*`: `*packmule*` also matches the pre-commit hook (and the npm install line). PID reuse: 86 PIDs on WS-BUILD-01 belong to more than one process over the week, 5612 included (it comes back days later as a SYSTEM `svchost.exe`). None of these searches trips on it because each one carries `hostname=` and the children all come from the same few minutes. If someone drops `hostname=`, that's the lesson.

A4. What did that shell run?

Why this pivot: Commands tell you intent. Follow the `/k` shell's children: its `pid` from A3 goes in as the `ppid`.

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" ppid=<the /k shell's pid>
| sort _time
| table _time pid ppid properties.parent_image_path principal
  properties.image_path properties.command_line
```

Guiding question (interpretation): *You have a chain of four searches. What's the one-sentence story, and why does it matter that the package is signed with a clean hash?*

> **Answer key — staff + self-study**
>
> **Type:** `ppid=5668` Expected: three rows within ~12s, starting ~14s after the shell opened: `whoami /all` (pid 5688), `net group "Domain Admins" /domain` (5696), `ipconfig /all` (5704), each with parent `C:\Windows\System32\cmd.exe`: host and domain recon (TIER_MANIFEST A3). Model answer (interpretation Q): install → post-install script → interactive shell → recon, all under alice's PowerShell in under 3 min. The binary is signed with a clean hash and no CVE, so indicators are useless. Only the behavior gives it away. This is the shape Module 2 reuses.

Part B — Count the cost (3 min)

Guiding question (reflection): *How many searches did that take? How many PIDs did you write down and retype, how many tabs are open, how many notes did you scribble? And where is any of it written down right now?*

Count them out loud. Then look at where the answer lives: your browser history and your head. Close the laptop and the hunt is gone.

> **Answer key — staff + self-study**
>
> Model answer (reflection Q): four searches, two PIDs retyped (5648, 5668) and one `ppid` compared by eye (5612), each a chance to get a digit wrong, at least one tab per search, and no written record. That's four hops for one process tree on one host. A real hunt runs 20+ searches. This count is the number the M3/M4 demo pays back: the agent follows the same chain, writes the hunt down as it goes, and takes minutes. Part C fixes the "written down" half by hand.

Part C — Write it down (10 min)

Guiding question (decision): *You found one whole chain. You're about to find a loose thread. Which hunt is `completed` and which stays `in-progress`, and why is writing down the open one just as important?*

H-0004 — the post-install execution chain

Six steps, one slide each.

1. Create the hunt. Copy the whole command and put your name in `--hunter`.

```
athf hunt new \
  --title "Trojanized build-CLI post-install spawns shell + recon" \
  --technique T1195.002 \
  --tactic execution --tactic initial-access \
  --platform windows \
  --data-source edr \
  --actor "CRON GOBLIN" \
  --behavior "package-manager post-install script is followed by an interactive shell that runs host/domain recon" \
  --location "developer workstations (WS-BUILD-*, WS-DEV-*)" \
  --evidence "eCAR process create, linked by pid/ppid on one host: node postinstall.js, then a sibling cmd.exe /k shell running whoami/net/ipconfig" \
  --threat-context "packmule-cli@2.4.1 backdoored via compromised publish token; signed binary, clean hash, no CVE" \
  --hunter "<your-name>" \
  --non-interactive
```

2. Open the file. ATHF names it with the next free number (**H-0004** in a fresh workspace, after the three examples) and prints the path: `hunts/production/2026/Q4/H-0004.md`. Open that file in your editor. The **ABLE Scoping** table under LEARN is already filled from your flags.

> **Tip (macOS):** `open -t hunts/production/2026/Q4/H-0004.md` pops the file straight into your text editor from the terminal.

> **If you're ahead:** ATHF's template puts the *Hypothesis Statement* under **LEARN**, and calls **OBSERVE** *Expected Behaviors*. Type the hypothesis: *CRON GOBLIN uses a post-install script to open a recon shell on dev workstations.* Under **OBSERVE** → *What Suspicious Looks Like*, type: *a post-install script, then an interactive `/k` shell from the same parent running recon.*

3. Paste CHECK. Two blocks, both under **CHECK → Hunting Queries**.

H-0004 CHECK, queries. Paste over `[Your initial query]` and its fences.

~~~~spl
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" properties.command_line="*postinstall.js*"

index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" ppid=5648

index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" properties.image_path="*\\cmd.exe"
properties.command_line="*telemetry*"
| sort _time
| table _time pid ppid properties.parent_image_path principal
  properties.image_path properties.command_line

index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" ppid=5668
| sort _time
| table _time pid ppid properties.parent_image_path principal
  properties.image_path properties.command_line
~~~~

H-0004 CHECK, notes. Select the `**Query Notes:**` line and its two placeholder bullets, and paste this over them.

```
**Query Notes:**
- A1: node postinstall.js as alice.chmod. pid 5648, ppid 5612 (PowerShell).
- A2: node's only child: cmd.exe /c "packmule --activate-telemetry". Dead end.
- A3: cmd.exe /k "packmule telemetry shell", pid 5668, ppid 5612: node's sibling.
- A4: the /k shell ran whoami /all, net group "Domain Admins" /domain, ipconfig /all.
```

4. Paste KEEP. Two blocks, both under **KEEP**.

H-0004 KEEP. Select from the `[Concise summary…]` line down to the second example row of the **Findings** table (the `suspected` one), and paste this over it. The evidence-gate line under the table stays.

```
**Suspected.** Initial access + execution on WS-BUILD-01 (alice.chmod).

### Findings

Things you're reporting as malicious. Only `confirmed` and `suspected` belong here.

| **Verdict** | **Subject** | **Ticket** | **Evidence** | **Independent Confirmation** |
|-------------|-------------|-----------|--------------|------------------------------|
| `suspected` | WS-BUILD-01 · alice.chmod | N/A | postinstall → /k shell → recon | None — telemetry only |
```

H-0004 FOLLOW-UP. Under **Follow-up Actions**, replace the two example lines with:

```
- [ ] Outbound C2 from WS-BUILD-01 → H-0005
```

5. Set status and verdict. Change `planning` to `completed` in two places: the `status:` line at the top of the file and the `- **Status:**` line under Hunt Metadata. The verdict, `suspected`, came in with the KEEP block.

> **On "verdict":** each **Findings** row has one. Read the line under the table: `confirmed` needs proof from outside the logs (host forensics, a reproduction). Logs alone are `suspected`, and that's all we have today. **Status** is separate: it says whether you're done hunting (`planning` → `in-progress` → `completed`).

6. Validate.

```
athf hunt validate H-0004
```

Expect ✅ `Hunt is valid!`

> **Answer key — staff + self-study**
>
> Target shape: `HUNTS_EXPECTED/H-0004.md` (same file shape as the attendee's), which staff can project next to theirs. Check `status: completed`, the four queries with `ppid=5648` and `ppid=5668`, one `suspected` Findings row for `WS-BUILD-01` / `alice.chmod`. (ATHF 0.24's evidence gate: telemetry alone never reaches `confirmed`.) Attendees who paste only the blocks still have template placeholders under Query Performance, Ruled Out and Lessons Learned; `athf hunt validate` passes anyway. The expected file fills them for staff. Hint trigger: if a paste lands in the wrong place, the usual cause is selecting past `### Query Performance` or past the Findings table. Undo and reselect. Model answer (decision Q): H-0004 is `completed` (you followed the whole chain). H-0005 stays `in-progress` (a beacon observed but not explained). Both verdicts are `suspected`: logs only. Writing down the open one matters because it's the thread Module 3 pulls. An honest "not done yet" is a real deliverable.

H-0005 — where did it call home?

Endpoint logs show only the proxy (kickoff slide: *How traffic leaves*). So you get the real destination in three hops: **host → proxy → domain**.

C1. Where did node connect to?

Why this pivot: Find the network connections node opened on this host, which process opened them (`pid`), and this host's own IP.

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=FLOW
hostname="WS-BUILD-01" properties.direction=OUTBOUND properties.image_path="*node*"
| stats count by pid properties.src_ip properties.dst_ip properties.dst_port
```

`properties.direction=OUTBOUND` keeps only the connections this host started. Without it, servers log *inbound* flows from the same IP, so one IP looks like many hosts.

Compare the `pid` with node's pid from A1. Then write down two values: `src_ip` is **WS-BUILD-01's IP**, and `dst_ip:dst_port` is where node went.

> **Answer key — staff + self-study**
>
> Expected: one row. `pid` **5996**, `src_ip=10.60.10.11` → `10.60.20.30:3128`, count **121**. That's PROXY-01, matching the kickoff diagram, so the real destination isn't here. (Without the OUTBOUND filter, `src_ip=10.60.10.11` shows up on 18 hosts' inbound flows.) pid 5996 is not 5648: these flows come from a later node process, `node ...\AppData\packmule\hooks\loader.js --beacon` (the flow events carry its command line too), started ~75 min after the install. Its first flow is ~90 min after install. Same host, and that's what we follow. How that loader got there is persistence, which the M3 demo covers. Don't chase it here.

C2. What did the proxy connect to for this host?

Why this pivot: The proxy logs the client IP and the destination. Search for WS-BUILD-01's IP from C1. `http_method=CONNECT` keeps only HTTPS tunnels (what a script like node opens) and drops the inspected browser traffic. Next to each destination you also get `url_category` (the proxy's label for the site), `user` (the login the proxy saw) and `bytes_out`.

```
index=thrunt sourcetype="apache:access:json" client="<the IP from C1>"
http_method=CONNECT
| stats count, sum(bytes_out) AS bytes_out by server, url_category, user
| sort - count
```

Which of these names don't you recognize? Compare its category and user with the well-known name above it, and add up its rows against C1's count.

> **Answer key — staff + self-study**
>
> **Type:** `client="10.60.10.11"` Expected top rows (server · url_category · user · count · bytes_out):
>
> - `ctldl.windowsupdate.com` · Software/Updates · – · 158 · 36396
> - `cdn-packmule-telemetry.net` · Software/Updates · alice.chmod · **103** · 23461
> - `login.microsoftonline.com` · Business/Economy · alice.chmod · 62 · 14509
>
> (`user` reads `SUDOSHIPPING\alice.chmod`.) Further down, `cdn-packmule-telemetry.net` has two `Blocked` rows, 13 as alice.chmod and 5 as –: the proxy refused 18 tunnels. 103 + 13 + 5 = **121**, exactly node's flow count from C1. What the extra fields say: `url_category` is `Software/Updates`, the same as Windows Update, so the beacon is dressed as update traffic. `bytes_out` doesn't separate them either: ~230 bytes per tunnel for both, because a CONNECT log only counts the handshake. The tell is `user`: real Windows Update runs as the machine (–), and this "update" runs under alice's login. Without `http_method=CONNECT` the domain's top row drops to #12, under Microsoft/Google browsing, which is why the filter is there.

C3. Is it regular?

Why this pivot: Malware calls home on a timer. People don't.

```
index=thrunt sourcetype="apache:access:json" client="<the IP from C1>"
server="<the name you didn't know>"
| timechart span=5m count
```

> **Answer key — staff + self-study**
>
> **Type:** `client="10.60.10.11"` and `server="cdn-packmule-telemetry.net"` Expected: a steady 0–2 per 5-min bucket for ~10h, never a long gap (121 requests, median gap ~300s, 200–400s jitter), starting the same second as node's first flow in C1. That's a beacon. Scoped to one client on purpose: dropping `client=` also shows `10.60.30.13` (CI-RUNNER-03), which is the M4 "same actor, new place" reveal. Don't show it here.

Now record it as **H-0005**, the same six steps as H-0004.

1. Create the hunt. Copy the whole command and put your name in `--hunter`.

```
athf hunt new \
  --title "Periodic outbound to CDN-lookalike after build-CLI install" \
  --technique T1071.001 \
  --tactic command-and-control \
  --platform windows \
  --data-source proxy \
  --actor "CRON GOBLIN" \
  --behavior "regular-interval HTTPS to a young CDN-lookalike domain following post-install execution" \
  --location "WS-BUILD-01 egress via PROXY-01" \
  --evidence "eCAR FLOW to the proxy, then proxy access log: ~5-min-interval CONNECTs to cdn-packmule-telemetry.net" \
  --hunter "<your-name>" \
  --non-interactive
```

2. Open the file: `hunts/production/2026/Q4/H-0005.md`, next to H-0004 (macOS: `open -t hunts/production/2026/Q4/H-0005.md`).

3. Paste CHECK. Same two placeholders as H-0004.

H-0005 CHECK, queries. Paste over `[Your initial query]` and its fences.

```
~~~spl
index=thrunt sourcetype="evidenceforge:ecar:json" object=FLOW
hostname="WS-BUILD-01" properties.direction=OUTBOUND properties.image_path="*node*"
| stats count by pid properties.src_ip properties.dst_ip properties.dst_port

index=thrunt sourcetype="apache:access:json" client="10.60.10.11" http_method=CONNECT
| stats count, sum(bytes_out) AS bytes_out by server, url_category, user
| sort - count

index=thrunt sourcetype="apache:access:json" client="10.60.10.11"
server="cdn-packmule-telemetry.net"
| timechart span=5m count
~~~
```

H-0005 CHECK, notes. Select the `**Query Notes:**` line and its two bullets, and paste this over them.

```
**Query Notes:**
- C1: pid 5996 (a later node), 10.60.10.11 → 10.60.20.30:3128 (PROXY-01), 121 flows.
- C2: cdn-packmule-telemetry.net, "Software/Updates" as alice: 103 + 18 blocked = 121.
- C3: steady 0–2 per 5-min bucket for ~10h, never a long gap.
```

4. Paste KEEP. Same selections as H-0004.

H-0005 KEEP. Select from `[Concise summary…]` down to the `suspected` example row, and paste this over it.

```
**Suspected, still open.** ~5-min beacon to cdn-packmule-telemetry.net via PROXY-01.

### Findings

Things you're reporting as malicious. Only `confirmed` and `suspected` belong here.

| **Verdict** | **Subject** | **Ticket** | **Evidence** | **Independent Confirmation** |
|-------------|-------------|------------|--------------|------------------------------|
| `suspected` | WS-BUILD-01 · 10.60.10.11 | N/A | ~5-min CONNECTs, ~10h | None — telemetry only |
```

H-0005 FOLLOW-UP. Replace the two example lines under **Follow-up Actions**:

```
- [ ] Who else talks to it? Real CDN?
```

5. Set status and verdict. Change `planning` to `in-progress` in both status lines: the hunt stays open. The verdict stays `suspected` (logs only); the follow-up line is where "needs follow-up" lives.

6. Validate.

```
athf hunt validate H-0005
```

Expect ✅ `Hunt is valid!`

> **Answer key — staff + self-study**
>
> Target shape: `HUNTS_EXPECTED/H-0005.md`. `status: in-progress`, one `suspected` Findings row. (ATHF 0.24's status words are planning / in-progress / completed; its verdicts are confirmed / suspected / attempted_not_vulnerable / benign / inconclusive. "Still open" is the status, not a verdict. "needs-follow-up" is not an ATHF verdict, and ATHF treats "follow-up required"-style text as a placeholder, so the open thread lives in the status and the Follow-up Actions line.) This open thread is what Module 3 picks up (shared-infrastructure pivot to the second domain).

Stop here — you should see

```
athf hunt list
```

H-0004 and H-0005, plus the 3 examples. Widen your terminal if the ID column shows `H…`.

```
athf hunt validate
```

Every hunt passes.

Fell behind? Copy the two `athf hunt new` commands above, then the paste blocks, and you're caught up.

Next: that was one host and four hops, and it only survives because you wrote it down. In Module 2 you search those notes to find a victim you had no indicator for.